

MASTERSCOPE

MasterScope is an interactive program for analyzing and cross referencing user programs. It contains facilities for analyzing user functions to determine what other functions are called; how and where variables are bound, set, or referenced; and which functions use particular record declarations. MasterScope can analyze definitions directly from a file as well as in-memory definitions.

MasterScope maintains a database of the results of the analyses it performs. Via a simple command language, you may interrogate the database, call the editor on those expressions in functions that were analyzed which use variables or functions in a particular way, or display the tree structure of function calls among any set of functions.

MasterScope is interfaced with the editor and file manager so that when a function is edited or a new definition loaded in, MasterScope knows that it must reanalyze that function.

With the Medley release, MasterScope now understands Common Lisp **defun**, **defmacro**, and **defvar**.

Requirements

MSANALYZE, MSPARSE, MSCOMMON, MS-PACKAGE

You may also want to make use of Browser, DataBaseFns, and SEdit or DEdit.

Installation

Load `MASTERSCOPE.DFASL` and the other `.DFASL` files from the library.

MasterScope Command Language

You communicate with MasterScope using an English-like command language, e.g., `WHO CALLS PRINT`. With these commands, you can direct that functions be analyzed, interrogate the MasterScope database, and perform other operations. The commands deal with sets of functions, variables, etc., and relations between them (e.g., `call`, `bind`). Sets correspond to English nouns; relations correspond to verbs.

A set of atoms can be specified in a variety of ways, either explicitly, e.g., `FUNCTIONS ON FIE` specifies the atoms in `(FILEFNLSLT 'FIE)`, or implicitly, e.g., `NOT CALLING Y`, where the meaning must be determined in the context of the rest of the command. Such sets of atoms are the basic building blocks with which the command language deals.

MasterScope also deals with relations between sets.

For example, the relation `CALL` relates functions and other functions; the relations `BIND` and `USE FREELY` relate functions and variables. These relations get stored in the MasterScope database when functions are analyzed. In addition, MasterScope "knows" about file manager conventions; `CONTAIN` relates files and various types of objects (functions, variables).

Sets and relations are used (along with a few additional words) to form sentence-like commands.

For example, the command `WHO ON 'FOO USE 'X FREELY` prints out the list of functions contained in the file `FOO` which use the variable `X` freely. The command `EDIT WHERE ANY CALLS 'ERROR` calls `EDITF` (see *IRM*) on those functions which have previously been analyzed that directly call `ERROR`, pointing at each successive expression where the call to `ERROR` actually occurs.

MasterScope Commands

The normal mode of communication with MasterScope is via commands. These are sentences in the MasterScope command language which direct MasterScope to answer questions or perform various operations.

MasterScope commands are typed into the Executive window, preceded by a period (.) to distinguish them from other commands to the Exec. MasterScope keywords can be in any package, so MasterScope commands can be issued in any type of Exec. The commands may be typed uppercase or lowercase.

To use a keyword as a variable or function name, you must use a single quote in front of it, e.g., `.WHO SETS 'SETS`.

Note: Any MasterScope command may be followed by `OUTPUT FILENAME` to send output to the given file rather than the terminal, e.g., `.WHO CALLS WHO OUTPUT CROSSREF`.

`ANALYZE SET`

[MasterScope command]

Analyzes the functions in *SET* (and any functions called by them) and includes the information gathered in the database. MasterScope does not reanalyze a function if it thinks it already has valid information about that function in its database. You may use the command `REANALYZE` to force reanalysis.

Note that whenever a function is referred to in a command as a subject of one of the relations, it is automatically analyzed; you need not give an explicit `ANALYZE` command. Thus, `WHO IN MYFNS CALLS FIE` automatically analyzes the functions in `MYFNS` if they have not already been analyzed.

Note also that only `EXPR` definitions are analyzed; that is, MasterScope does not analyze compiled code. If necessary, the definition is `DWIMIFYed` before analysis. If there is no in-core definition for a function (either in the function definition cell or an `EXPR` property), MasterScope attempts to read in the definition from a file. Files which have been explicitly mentioned previously in some command are searched first. If the definition cannot be found on any of those files, MasterScope looks among the files on `FILELST` for a definition. If a function is found in this manner, MasterScope prints a message "(reading from *FILENAME*)". If no definition can be found at all, MasterScope prints a message "*FN* can't be analyzed". If the function previously was known, the message "*FN* disappeared!" is printed.

`REANALYZE SET`

[MasterScope command]

Causes MasterScope to reanalyze the functions in *SET* (and any functions called by them) even if it already has valid information in its database. This would be necessary if you had disabled or subverted the file manager; e.g., performed `PUTD`'s to change the definition of functions.

ERASE *SET* [MasterScope command]

Erases all information about the functions in *SET* from the database. ERASE by itself clears the entire database.

SHOW PATHS *PATHOPTIONS* [MasterScope command]

Displays a tree of function calls. This is described fully in "SHOW PATHS" below.

SET RELATION SET [MasterScope command]

SET IS SET [MasterScope command]

SET ARE SET [MasterScope command]

These commands have the same format as an English sentence with a subject (the first *SET*), a verb (*RELATION* or *IS* or *ARE*), and an object (the second *SET*). Any of the *SET*'s within the command may be preceded by the question determiners *WHICH* or *WHO* (or just *WHO* alone).

For example, *WHICH FUNCTIONS CALL X* prints the list of functions that call the function *X*.

RELATION may be one of the relation words in present tense (*CALL*, *BIND*, *TEST*, *SMASH*, etc.) or used as a passive (e.g., *WHO IS CALLED BY WHO*). Other variants are allowed, e.g., *WHO DOES X CALL*, *IS FOO CALLED BY FIE*, etc.

The interpretation of the command depends on the number of question elements present:

If there is no question element, the command is treated as an assertion and MasterScope returns either *T* or *NIL*, depending on whether that assertion is true. Thus, *ANY IN MYFNS CALL HELP* prints *T* if any function in *MYFNS* call the function *HELP*, and *NIL* otherwise.

If there is one question element, MasterScope returns the list of items for which the assertion would be true.

For example,

```
MYFN BINDS WHO USED FREELY BY YOURFN
```

prints the list of variables bound by *MYFN* which are also used freely by *YOURFN*.

If there are two question elements, MasterScope prints a doubly indexed list:

```
_. WHO CALLS WHO IN /FNS
RECORDSTATEMENT -- /RPLNODE
RECORDECL1 -- /NCONC, /RPLACD, /RPLNODE
RECREDECLARE1 -- /PUTHASH
UNCLISPTRAN -- /PUTHASH, /RPLNODE2
RECORDWORD -- /RPLACA
RECORD1 -- /RPLACA, /SETTOPVAL
EDITREC -- /SETTOPVAL
```

EDIT WHERE *SET RELATION SET* [- *EDITCOMS*] [MasterScope command]

(WHERE may be omitted.) The first *SET* refers to a set of functions. The *EDIT* command calls the editor on each expression where the *RELATION* actually occurs.

For example, `EDIT WHERE ANY CALL ERROR` calls `EDITF` on each (analyzed) function which calls `ERROR` stopping within a TTY: at each call to `ERROR`. Currently you cannot `EDIT WHERE` a file which CONTAINS a datum, nor where one function CALLS another SOMEHOW.

EDITCOMS, if given, is a list of commands passed to `EDITF` to be performed at each expression.

For example,

```
EDIT WHERE ANY CALLS MYFN DIRECTLY - (SW 2 3) P
```

switches the first and second arguments to `MYFN` in every call to `MYFN` and prints the result. `EDIT WHERE ANY ON MYFILE CALL ANY NOT @ GETD` calls the editor on any expression involving a call to an undefined function.

Note that `EDIT WHERE X SETS Y` points only at those expressions where `Y` is actually set, and skips over places where `Y` is otherwise mentioned.

`SHOW WHERE SET RELATION SET` [MasterScope command]

Like the `EDIT` command except merely prints out the expressions without calling the editor.

`EDIT SET [- EDITCOMS]` [MasterScope command]

Calls `EDITF` on each function in *SET*. *EDITCOMS*, if given, is passed as a list of editor commands to be Executed.

For example,

```
EDIT ANY CALLING FN1 - (R FN1 FN2)
```

replaces `FN1` by `FN2` in those functions that call `FN1`.

`DESCRIBE SET` [MasterScope command]

Prints the `BIND`, `USE FREELY` and `CALL` information about the functions in *SET*.

For example, the command `DESCRIBE PRINTARGS` might print out:

```
PRINTARGS [N, FLG]
  binds:      TEM, LST, X
  calls:      MSRECORDFILE, SPACES, PRIN1
  called by:  PRINTSENTENCE, MSHELP, CHECKER
```

This shows that `PRINTARGS` has two arguments, `N` and `FLG`; binds internally the variables `TEM`, `LST` and `X`; calls `MSRECORDFILE`, `SPACES` and `PRIN1`; and is called by `PRINTSENTENCE`, `MSHELP`, and `CHECKER`.

You can specify additional information to be included in the description. `DESCRIBELST` is a list each of whose elements is a list containing a descriptive string and a form. The form is evaluated (it can refer to the name of the function being described by the free variable `FN`). If it returns a non-NIL value, the description string is printed followed by the value. If the value is a list, its elements are printed with commas between them.

For example, the entry

```
("types:  " (GETRELATION FN ' (USE TYPE) T)
```

would include a listing of the types used by each function.

CHECK *SET*

[MasterScope command]

Checks for various anomalous conditions (mainly in the compiler declarations) for the files in *SET* (if *SET* is not given, *FILELST* is used).

For example, this command warns about:

- Variables which are bound but never referenced
- Functions in *BLOCKS* declarations which aren't on the file containing the declaration
- Functions declared as *ENTRIES* but not in the block
- Variables which may not need to be declared *SPECVARS* because they are not used freely below the places where they are bound

FOR *VARIABLE SET I.S.TAIL*

[MasterScope command]

This command provides a way of combining *CLISP* iterative statements with MasterScope. An iterative statement is constructed in which *VARIABLE* is iteratively assigned to each element of *SET*, and then the iterative statement tail *I.S.TAIL* is executed.

For example,

```
FOR X CALLED BY FOO WHEN CCODEP DO (PRINTOUT T X , , ,  
(ARGLIST X) T)
```

prints out the name and argument list of all of the compiled functions which are called by *FOO*.

MasterScope Relations

A relation is specified by one of the keywords below. Some of these "verbs" accept modifiers.

For example, *USE*, *SET*, *SMASH* and *REFERENCE* all may be modified by *FREELY*. The modifier may occur anywhere within the command. If there is more than one verb, any modifier between two verbs is assumed to modify the first one.

For example, in

```
USING ANY FREELY OR SETTING X,
```

FREELY modifies *USING* but not *SETTING*. The entire phrase is interpreted as the set of all functions which either use any variable freely or set the variable *X*, whether or not *X* is set freely. Verbs can occur in the present tense (e.g., *USE*, *CALLS*, *BINDS*, *USES*) or as present or past participles (e.g., *CALLING*, *BOUND*, *TESTED*). The relations (with their modifiers) recognized by MasterScope are:

CALL

[MasterScope relation]

Function *F1* calls *F2* if the definition of *F1* contains a form (*F2 --*). The *CALL* relation also includes any instance where a function uses a name as a function, as in

```
(APPLY (QUOTE F2) --), (FUNCTION F2), etc.
```

(*CALL* and *CALLS* are equivalent.)

CALL SOMEHOW [MasterScope relation]

One function calls another SOMEHOW if there is some path from the first to the other. That is, if F1 calls F2, and F2 calls F3, then F1 CALLS F3 SOMEHOW.

This information is not stored directly in the database; instead, MasterScope stores only information about direct function calls, and (re)computes the CALL SOMEHOW relation as necessary.

USE [MasterScope relation]

If unmodified, the relation USE denotes variable usage in any way; it is the union of the relations SET, SMASH, TEST, and REFERENCE.

SET [MasterScope relation]

A function SETs a variable if the function contains a form

(SETQ var --), (SETQ var --), etc.

SMASH [MasterScope relation]

A function SMASHes a variable if the function calls a destructive list operation (RPLACA, RPLACD, DREMOVE, SORT, etc.) on the value of that variable.

MasterScope also finds instances where the operation is performed on a part of the value of the variable. For example, if a function contains a form (RPLACA (NTH X 3) T), it is noted as SMASHing X.

If the function contains a sequence (SETQ Y X), (RPLACA Y T), then Y is noted as being SMASHed, but not X.

TEST [MasterScope relation]

A variable is TESTed by a function if its value is only distinguished between NIL and non-NIL.

For example, the form (COND ((AND X --) --)) tests the value of X.

REFERENCE [MasterScope relation]

This relation includes all variable usage except for SET.

Note: The verbs USE, SET, SMASH, TEST and REFERENCE may be modified by the words FREELY or LOCALLY. A variable is used FREELY if it is not bound in the function at the place of its use. It is used LOCALLY if the use occurs within a PROG or LAMBDA that binds the variable.

MasterScope also distinguishes between CALL DIRECTLY and CALL INDIRECTLY. A function is called directly if it occurs as CAR-of-form in a normal evaluation context. A function is called indirectly if its name appears in a context which does not imply its immediate evaluation, for example (SETQ Y (LIST (FUNCTION FOO) 3)). The distinction is whether or not the compiled code of the caller would contain a direct call to the callee.

Note that an occurrence of (FUNCTION FOO) as the functional argument to one of the built-in mapping functions which compile open is considered to be a direct call.

In addition, CALL FOR EFFECT (where the value of the function is not used) is distinguished from CALL FOR VALUE.

BIND	[MasterScope relation]
The BIND relation between functions and variables includes both variables bound as function arguments and those bound in an internal PROG or LAMBDA expression.	
USE AS A FIELD	[MasterScope relation]
MasterScope notes all uses of record field names within FETCH, REPLACE or CREATE expressions.	
FETCH	[MasterScope relation]
Use of a field within a FETCH expression.	
REPLACE	[MasterScope relation]
Use of a record field name within a REPLACE or CREATE expression.	
USE AS A RECORD	[MasterScope relation]
MasterScope notes all uses of record names within CREATE or TYPE? expressions. Additionally, in (fetch (FOO FIE) of X), FOO is used as a record name.	
CREATE	[MasterScope relation]
Use of a record name within a CREATE expression.	
USE AS A PROPERTY NAME	[MasterScope relation]
MasterScope notes the property names used in expressions such as GETPROP, PUTPROP, GETLIS, etc., if the name is quoted; e.g. if a function contains a form (GETPROP X (QUOTE INTERP)), then that function USES INTERP as a property name.	
USE AS A CLISP WORD	[MasterScope relation]
MasterScope notes all iterative statement operators and user defined CLISP words as being used as a CLISP word.	
CONTAIN	[MasterScope relation]
Files CONTAIN functions, records, and variables. This relation is not stored in the database but is computed using the file manager.	
DECLARE AS LOCALVAR	[MasterScope relation]
DECLARE AS SPECVAR	[MasterScope relation]
MasterScope notes internal calls to DECLARE from within functions.	
ACCEPT	[MasterScope relation]
SPECIFY	[MasterScope relation]
KEYCALL	[MasterScope relation]
MasterScope notes keyword arguments of Common Lisp functions when they are analyzed and when they are called.	

FOO ACCEPTS :BAR is true if FOO is a Common Lisp function that accepts the keyword :BAR. **FOO ACCEPTS &ALLOW-OTHER-KEYS** is true if FOO has &ACCEPT-OTHER-KEYS in its lambda list.

FOO SPECIFIES :BAR is true if FOO is a function that calls any function with the keyword :BAR; the function in question must **ACCEPT :BAR**.

FOO KEYCALLS BAR is true if FOO is a function and calls BAR with one or more keywords it **ACCEPTS**.

FLET	[MasterScope relation]
LABEL	[MasterScope relation]
MACROLET	[MasterScope relation]
LOCAL-DEFINE	[MasterScope relation]

MasterScope tracks uses of Common Lisp local definition forms (it currently does not expand them while analyzing them, however).

FOO FLETS BAR is true if FOO is a function with a FLET defining BAR local to FOO.

LABELS and MACROLETS are similar. **LOCAL-DECLARES** is the union of FLETS, LABELS, and MACROLETS.

Abbreviations

The following abbreviations are recognized:

FREE=FREELY
 LOCAL=LOCALLY
 PROP=PROPERTY
 REF=REFERENCE

Also, the words A, AN and NAME (after AS) are "noise" words and may be omitted.

MasterScope Templates

MasterScope uses templates (see "Effecting MasterScope Analysis" below) to decide which relations hold between functions and their arguments.

For example, the information that **SORT** SMASHes its first argument is contained in the template for **SORT**. MasterScope initially contains templates for most system functions which set variables, test their arguments, or perform destructive operations. You may change existing templates or insert new ones in MasterScope's tables via the **SETTEMPLATE** function (below).

MasterScope also constructs templates to handle Common Lisp functions with keyword arguments. These constructed templates are noticed by **FILES?** and can be saved if desired, or MasterScope can recreate them by analyzing the functions again.

MasterScope Set Specifications

A set is a collection of things (functions, variables, etc.). A set is specified by a set phrase, consisting of a determiner (e.g., ANY, WHICH, WHO) followed by a type (e.g., FUNCTIONS, VARIABLES) followed by a specification (e.g., IN MYFNS). The determiner, type and specification may be used alone or in combination.

For example,

```
ANY FUNCTIONS IN MYFNS,  
VARIABLES IN GLOBALVARS, and  
WHO
```

are all acceptable set phrases.

Note: Sets may also be specified with relative clauses introduced by the word *THAT*, e.g. *THE FUNCTIONS THAT BIND 'X*.

'ATOM [MasterScope set specification]

The simplest way to specify a set consisting of a single thing is by the name of that thing.

For example, in the command *WHO CALLS 'ERROR*, the function *ERROR* is referred to by its name. Although the *'* (apostrophe) can be left out, to resolve possible ambiguities names should usually be quoted; e.g., *WHO CALLS 'CALLS* returns the list of functions which call the function *CALLS*.

'LIST [MasterScope set specification]

Sets consisting of several atoms may be specified by naming the atoms.

For example, the command *WHO USES ' (A B)* returns the list of functions that use the variables *A* or *B*.

IN EXPRESSION [MasterScope set specification]

The form *EXPRESSION* is evaluated, and its value is treated as a list of the elements of a set.

For example, *IN GLOBALVARS* specifies the list of variables in the value of the variable *GLOBALVARS*.

@ PREDICATE [MasterScope set specification]

A set may also be specified by giving a predicate which the elements of that set must satisfy. *PREDICATE* is either a function name, a *LAMBDA* expression, or an expression in terms of the variable *X*. The specification *@ PREDICATE* represents all atoms for which the value of *PREDICATE* is non-NIL.

For example, *@ EXPRP* specifies all those atoms which have *EXPR* definitions; *@ (STRPOS X CLISPCHARRAY)* specifies those atoms which contain *CLISP* characters. The universe to be searched is either determined by the context within the command (e.g., in *WHO IN FOOFNS CALLS ANY NOT @ GETD*, the predicate is only applied to functions which are called by any functions in the list *FOOFNS*), or in the extreme case, the universe defaults to the entire set of things which have been noticed by MasterScope, as in the command *WHO IS @ EXPRP*.

LIKE ATOM [MasterScope set specification]

ATOM may contain *ESCAPES*; it is used as a pattern to be matched, as in the editor.

For example, *WHO LIKE /R\$ IS CALLED BY ANY* would find both */RPLACA* and */RPLNODE*.

(The ESC character prints out as a \$; it is a wildcard for any number of characters.)

FIELDS OF *SET*

[MasterScope set specification]

SET is a set of records. This denotes the field names of those records.

For example, the command WHO USES ANY FIELDS OF BRECORD returns the list of all functions which do a fetch or replace with any of the field names declared in the record declaration of BRECORD.

KNOWN [MasterScope set specification]

The set of all functions which have been analyzed.

For example, the command `WHO IS KNOWN` prints out the list of functions which have been analyzed.

THOSE [MasterScope set specification]

The set of things printed out by the last MasterScope question.

For example, following the command

`WHO IS USED FREELY BY PARSE`

you could ask `WHO BINDS THOSE` to find out where those variables are bound.

ON PATH *PATHOPTIONS* [MasterScope set specification]

Refers to the set of functions which would be printed by the command `SHOW PATHS PATHOPTIONS`.

For example,

`IS FOO BOUND BY ANY ON PATH TO 'PARSE`

tests whether `FOO` might be bound above the function `PARSE` (that is, whether `FOO` is bound in any function that is higher up in the calling tree than `PARSE` is) . `SHOW PATHS` is explained in detail below.

Set Specifications by Relation

A set may also be specified by giving a relation its members must have with the members of another set:

RELATIONING SET [MasterScope set specification]

RELATIONING is used here generically to mean any of the relation words in the present participle form (possibly with a modifier), e.g., `USING`, `SETTING`, `CALLING`, `BINDING`. *RELATIONING SET* specifies the set of all objects which have that relation with some element of *SET*.

For example, `CALLING X` specifies the set of functions which call the function `X`; `USING ANY IN FOOVARS FREELY` specifies the set of functions which uses freely any variable in the value of `FOOVARS`.

RELATIONED BY SET [MasterScope set specification]

RELATIONED IN SET [MasterScope set specification]

This is similar to the *RELATIONING* construction.

For example, `CALLED BY ANY IN FOOFNS` represents the set of functions which are called by any element of `FOOFNS`; `USED FREELY BY ANY CALLING ERROR` is the set of variables which are used freely by any function which also calls the function `ERROR`.

Set Specifications by Blocktypes

BLOCKTYPE OF *FUNCTIONS*

[MasterScope set specification]

BLOCKTYPE ON *FILES*

[MasterScope set specification]

These phrases allow you to ask about BLOCKS declarations on files (see *IRM*).

BLOCKTYPE is one of LOCALVARS, SPECVARS, GLOBALVARS, ENTRIES, BLKFNS, BLKAPPLYFNS, or RETFNS.

BLOCKTYPE OF *FUNCTIONS* specifies the names which are declared to be *BLOCKTYPE* in any blocks declaration which contain any of *FUNCTIONS* (a "set" of functions). The "functions" in *FUNCTIONS* can either be block names or just functions in a block.

For example,

```
WHICH ENTRIES OF ANY CALLING 'Y BIND ANY GLOBALVARS ON
'FOO.
```

BLOCKTYPE ON *FILES* specifies all names which are declared to be *BLOCKTYPE* on any of the given *FILES* (a "set" of files).

Set Determiners

Set phrases may be preceded by a determiner, which is one of the words THE, ANY, WHO or WHICH. The question determiners (WHO and WHICH) are meaningful in only some of the commands, namely those that take the form of questions. ANY and WHO (or WHOM) can be used alone; they are wild-card elements, e.g., the command WHO USES ANY FREELY, prints out the names of all (known) functions which use any variable freely. If the determiner is omitted, ANY is assumed; e.g., the command WHO CALLS ' (PRINT PRIN1 PRIN2) prints the list of functions which call any of PRINT, PRIN1, PRIN2. THE is also allowed, e.g., WHO USES THE RECORD FIELD FIELDX.

Set Types

Any set phrase has a type; that is, a set may specify either functions, variables, files, record names, record field names or property names. The type may be determined by the context within the command (e.g., in CALLED BY ANY ON FOO, the set ANY ON FOO is interpreted as meaning the functions on FOO since only functions can be CALLED), or you may give the type explicitly (e.g., FUNCTIONS ON FIE).

The following types are recognized: FUNCTIONS, VARIABLES, FILES, PROPERTY NAMES, RECORDS, FIELDS, I.S.OPRS. Also, the abbreviations FNS, VARS, PROPNames or the singular forms FUNCTION, FN, VARIABLE, VAR, FILE, PROPNAME, RECORD, FIELD are recognized.

Note that most of these types correspond to built-in file manager types (see *IRM*).

The type is used by MasterScope in a variety of ways when interpreting the set phrase:

1. Set types are used to disambiguate possible parsings.

For example, both commands

```
WHO SETS ANY BOUND IN X OR USED BY Y
```

```
WHO SETS ANY BOUND IN X OR CALLED BY Y
```

have the same general form. However, the first case is parsed as

WHO SETS ANY (BOUND BY X OR USED BY Y)

since both BOUND BY X and USED BY Y refer to variables; while the second case is parsed as

WHO SETS ANY BOUND IN (X OR CALLED BY Y) ,

since CALLED BY Y and X must refer to functions.

Note that parentheses may be used to group phrases.

2. The type is used to determine the modifier for USE:

FOO USES WHICH RECORDS is equivalent to

FOO USES WHO AS A RECORD FIELD .

3. The interpretation of CONTAIN depends on the type of its object: the command

WHAT FUNCTIONS ARE CONTAINED IN MYFILE

prints the list of functions in MYFILE.

WHAT RECORDS ARE ON MYFILE

prints the list of records.

4. The implicit universe in which a set expression is interpreted depends on the type:

ANY VARIABLES @ GETD

is interpreted as the set of all variables which have been noticed by MasterScope (i.e., bound or used in any function which has been analyzed) that also have a definition.

ANY FUNCTIONS @ (NEQ (GETTOPVAL X) 'NOBIND)

is interpreted as the set of all functions which have been noticed (either analyzed or called by a function which has been analyzed) that also have a top-level value.

Conjunctions of Sets

Sets may be joined by the conjunctions AND and OR or preceded by NOT to form new sets. AND is always interpreted as meaning intersection; OR as union; NOT as complement.

For example, the set CALLING X AND NOT CALLED BY Y specifies the set of all functions which call the function X but are not called by Y.

Note: MasterScope's interpretation of AND and OR follow Lisp conventions rather than the conventional English interpretation.

"Calling X and Y" would, in English, be interpreted as the intersection of (CALLING X) and (CALLING Y); but MasterScope interprets CALLING X AND Y as CALLING ('X AND 'Y), which is the null set.

Only sets may be joined with conjunctions. Joining modifiers, as in

USING X AS A RECORD FIELD OR PROPERTY NAME

is not allowed; in this case, you must type

USING X AS A RECORD FIELD OR USING X AS A PROPERTY NAME

As described above, the type of set is used to disambiguate parsings. The algorithm used is to first try to match the type of the phrases being joined and then try to join with the longest preceding phrase.

In any case, you may group phrases with parentheses to specify the manner in which conjunctions should be parsed.

SHOW PATHS

In trying to work with large programs, you can lose track of the hierarchy of functions. The MasterScope `SHOW PATHS` command aids you by providing a map showing the calling structure of a set of functions. `SHOW PATHS` prints out a tree structure showing which functions call which other functions.

Loading the Browser library module modifies the `SHOW PATHS` command so the command's output is displayed as an undirected graph.

The `SHOW PATHS` command takes the form: `SHOW PATHS` followed by some combination of the following path options:

`FROM SET` [MasterScope path option]

Display the function calls from the elements of *SET*.

`TO SET` [MasterScope path option]

Display the function calls leading to elements of *SET*. If `TO` is given before `FROM` (or no `FROM` is given), the tree is inverted and a message (inverted tree) is printed to warn you that if `FN1` appears after `FN2` it is because `FN1` is called by `FN2`.

Note: When both `FROM` and `TO` are given, the first one indicates a set of functions to be displayed, while the second restricts the paths to be traced; i.e., the command `SHOW PATHS FROM X TO Y` traces the elements of the set CALLED SOMEHOW BY X AND CALLING Y SOMEHOW.

If `TO` is not given, `TO KNOWN OR NOT @ GETD` is assumed; that is, only functions which have been analyzed or which are undefined will be included.

Note that MasterScope analyzes a function while printing out the tree if that function has not previously been seen and it currently has an `EXPR` definition. Thus, any function which can be analyzed will be displayed.

`AVOIDING SET` [MasterScope path option]

Do not display any function in *SET*. `AMONG` is recognized as a synonym for `AVOIDING NOT`.

For example, `SHOW PATHS TO ERROR AVOIDING ON FILE2` does not display (or trace) any function on `FILE2`.

`NOTRACE SET` [MasterScope path option]

Do not trace from any element of *SET*. `NOTRACE` differs from `AVOIDING` in that a function which is marked `NOTRACE` is printed, but the tree beyond it is not expanded. The functions in an `AVOIDING` set are not printed at all.

For example,

`SHOW PATHS FROM ANY ON FILE1 NOTRACE ON FILE2`

displays the tree of calls emanating from `FILE1`, but does not expand any function on `FILE2`.

SEPARATE *SET*

[MasterScope path option]

Give each element of *SET* a separate tree.

Note: FROM and TO only insure that the designated functions are displayed. SEPARATE can be used to guarantee that certain functions begin new tree structures. SEPARATE functions are displayed in the same manner as overflow lines; i.e., when one of the functions indicated by SEPARATE is found, it is printed followed by a forward reference (a lowercase letter in braces) and the tree for that function is then expanded below.

LINELENGTH *N*

[MasterScope path option]

Resets LINELENGTH to *N* before displaying the tree. The linelength is used to determine when a part of the tree should "overflow" and be expanded lower.

Error Messages

When you give MasterScope a command, the command is first parsed, i.e. translated to an internal representation, and then the internal representation is interpreted.

If a command cannot be parsed, e.g. if you typed

```
SHOW WHERE CALLED BY X
```

MasterScope would reply

Sorry, I can't parse that!

and generate an error.

If the command is of the correct form but cannot be interpreted (e.g., the command EDIT WHERE ANY CONTAINS ANY) MasterScope prints the message

Sorry, that isn't implemented!

and generates an error.

If the command requires some functions having been analyzed (e.g., the command WHO CALLS X) and the database is empty, MasterScope prints the message

Sorry, no functions have been analyzed!

and generates an error.

Macro Expansion

As part of analysis, MasterScope expands the macro definition of called functions if they are not otherwise defined (see *IRM*). MasterScope always expands Common Lisp DEFMACRO definitions (unless it finds a template for the macro).

MasterScope Interlisp macro expansion is controlled by a variable:

MSMACROPROPS

[Variable]

Value is an ordered list of macro-property names that MasterScope searches to find a macro definition. Only the kinds of macros that appear on MSMACROPROPS are expanded. All others are treated as function calls and left unexpanded. Initially (MACRO).

Note: `MSMACROPROPS` initially contains only `MACRO` (not `10MACRO`, `DMACRO`, etc.) on the assumption that the machine-dependent macro definitions are more likely "optimizers".

If you edit a macro, MasterScope knows to reanalyze the functions which call that macro.

Note: If your macro is of the "computed-macro" style, and it calls functions which you edit, MasterScope does not notice. You must be careful to tell masterscope to `REANALYZE` the appropriate functions (e.g., if you edit `FOOEXPANDER` which is used to expand `FOO` macros, you have to `REANALYZE ANY CALLING FOO`.

Effecting MasterScope Analysis

MasterScope analyzes the `EXPR` definition of a function, and notes in its database the relations that this function has with other functions and with variables. To perform this analysis, MasterScope uses templates which describe the behavior of functions.

For example, the information that `SORT` destructively modifies its first argument is contained in the template for `SORT`. MasterScope initially contains templates for most system functions that set variables, test their arguments, or perform destructive operations.

A template is a list structure containing any of the following atoms:

`PPE` [in MasterScope template]

If an expression appears in this location, there is most likely a parenthesis error.

MasterScope notes this as a call to the function `ppe` (lowercase). Therefore, `SHOW WHERE ANY CALLS ppe` prints out all possible parenthesis errors. When MasterScope finds a possible parenthesis error in the course of analyzing a function definition, rather than printing the usual `"."`, it prints out a `"?"` instead. MasterScope notes functions called with keywords they do not accept as calls to `ppe`.

`NIL` [in MasterScope template]

The expression occurring at this location is not evaluated.

`SET` [in MasterScope template]

A variable appearing at this place is set.

`SMASH` [in MasterScope template]

The value of this expression is smashed.

`TEST` [in MasterScope template]

Is used as a predicate (that is, the only use of the value of the expression is whether it is `NIL` or non-`NIL`).

`PROP` [in MasterScope template]

Is used as a property name. If the value of this expression is of the form `(QUOTE ATOM)`, MasterScope notes that *ATOM* is USED AS A PROPERTY NAME.

For example, the template for GETPROP is `(EVAL PROP . PPE)`.

Must appear at the end of a template followed by the keywords the templated function accepts.

The expression at this point is used as a functional argument.

The expression at this location is evaluated (but not set, smashed, tested, used as a functional argument, etc.).

The value of the function (of which this is the template) is the value of this expression.

A combination of TEST and RETURN: If the value of the function is non-NIL, then it is returned. For instance, a one-element COND clause is this way.

The expression at this location is evaluated, but the value is not used. (That is, it is evaluated for its side effect only.)

An atom at this location is a field which is fetched.

An atom at this location is a field which is replaced.

An atom at this location is used as a record name.

An atom at this location is a record which is created.

An atom at this location is a variable which is bound.

CALL [in MasterScope template]

An atom at this location is a function which is called.

CLISP [in MasterScope template]

An atom at this location is used as a CLISP word.

! [in MasterScope template]

This atom, which can only occur as the first element of a template, allows you to specify a template for the CAR of the function form. If ! doesn't appear, the CAR of the form is treated as if it had a CALL specified for it. In other words, the templates (*.. EVAL*) and (*! CALL .. EVAL*) are equivalent.

If the next atom after a ! is NIL, this specifies that the function name should not be remembered.

For example, the template for AND is (*! NIL .. TEST RETURN*), which means that if you see an AND, don't remember it as being called. This keeps the MasterScope database from being cluttered by too many uninteresting relations. MasterScope also throws away relations for COND, CAR, CDR, and a couple of others.

Special Forms

In addition to the above atoms that occur in templates, there are some special forms which are lists keyed by their CAR.

.. TEMPLATE [in MasterScope template]

Any part of a template may be preceded by the atom *..* (two periods) which specifies that the template should be repeated an indefinite number ($N \geq 0$) of times to fill out the expression.

For example, the template for COND might be

(*.. (TEST .. EFFECT RETURN)*)

while the template for SELECTQ is

(*EVAL .. (NIL .. EFFECT RETURN) RETURN*).

(Although MasterScope "throws away" the relations for COND, it makes sense to template COND because there may be important information within the arguments of COND.)

(BOTH *TEMPLATE1 TEMPLATE2*) [in MasterScope template]

Analyze the current expression twice, using the each of the templates in turn.

(IF *EXPRESSION TEMPLATE₁ TEMPLATE₂*) [in MasterScope template]

Evaluate *EXPRESSION* at analysis time (the variable *EXPR* is bound to the expression which corresponds to the IF), and if the result is non-NIL, use *TEMPLATE₁*, otherwise *TEMPLATE₂*. If *EXPRESSION* is a literal atom, it is APPLYD to *EXPR*.

For example,

```
(IF LISTP (RECORD FETCH) FETCH)
```

specifies that if the current expression is a list, then the first element is a record name and the second element a field name, otherwise it is a field name.

(@ *EXPRFORM TEMPLATEFORM*) [in MasterScope template]

Evaluate *EXPRFORM* giving *EXPR*, evaluate *TEMPLATEFORM* giving *TEMPLATE*. Then analyze *EXPR* with *TEMPLATE*. @ lets you compute on the fly both a template and an expression to analyze with it. The forms can use the variable *EXPR*, which is bound to the current expression.

(MACRO . *MACRO*) [in MasterScope template]

MACRO is interpreted in the same way as macros (see *IRM*) and the resulting form is analyzed. If the template is the atom *MACRO* alone, MasterScope uses the *MACRO* property of the function itself. This is useful when analyzing code which contains calls to user-defined macros. If you change a macro property (e.g., by editing it) of an atom which has template of *MACRO*, MasterScope marks any function which used that macro as needing to be reanalyzed.

Some examples of templates:

Function	Template
DREVERSE	(SMASH . PPE)
AND	(! NIL TEST .. RETURN)
MAPCAR	(EVAL FUNCTION FUNCTION)
COND	(! NIL .. (IF CDR (TEST .. EFFECT RETURN) (TESTRETURN . PPE)))

Templates may be changed and new templates defined using the following functions:

(GETTEMPLATE *FN*) [Function]

Returns the current template of *FN*.

(SETTEMPLATE *FN TEMPLATE*) [Function]

Changes the template for the function *FN* and returns the old value. If any functions in the database are marked as calling *FN*, they are marked as needing reanalysis.

Updating the MasterScope Database

MasterScope is interfaced to the editor and file manager so that it notes whenever a function has been changed, either through editing or loading in a new definition. Whenever a command is given which requires knowing the information about a specific function, if that function has been noted as being changed, the function is automatically reanalyzed before the command is interpreted. If the command requires that all the information in the database be consistent (e.g., you ask `WHO CALLS X`) then all functions which have been marked as changed are reanalyzed.

MasterScope Entries

(MASTERSCOPE *COMMAND*—) [Function]

Top level entry to MasterScope. If *COMMAND* is *NIL*, enters into an Executive in which you may enter commands. If *COMMAND* is not *NIL*, the command is interpreted and MASTERSCOPE returns the value that would be printed by the command.

Note that only the question commands return meaningful values.

(CALLS *FN USEDATABASE*—) [Function]

FN can be a function name, a definition, or a form.

Note: *CALLS* also works on compiled code. *CALLS* returns a list of four elements:

- Functions called by *FN*
- Variables bound in *FN*
- Variables used freely in *FN*
- Variables used globally in *FN*

For the purpose of *CALLS*, variables used freely which are on *GLOBALVARS* or have a property *GLOBALVAR* value *T* are considered to be used globally. If *USEDATABASE* is *NIL* (or *FN* is not a symbol), *CALLS* performs a one-time analysis of *FN*. Otherwise (i.e., if *USEDATABASE* is non-*NIL* and *FN* a function name), *CALLS* uses the information in MasterScope's database (*FN* is analyzed first if necessary).

(CALLSCCODE *FN* —) [Function]

The subfunction of *CALLS* which analyzes compiled code. *CALLSCCODE* returns a list of elements:

- Functions called via "linked" function calls (not implemented in Interlisp-D)
- Functions called regularly
- Variables bound in *FN*
- Variables used freely
- Variables used globally

(FREEVARS *FN USEDATABASE*) [Function]

Equivalent to (CADDR (CALLS *FN USEDATABASE*)). Returns the list of variables used freely within *FN*.

(SETSYNONYM *PHRASE MEANING*—) [Function]

Defines a new synonym for MasterScope's parser. Both *OLDPHRASE* and *NEWPHRASE* are words or lists of words; anywhere *OLDPHRASE* is seen in a command, *NEWPHRASE* is substituted.

For example,

```
(SETSYNONYM 'GLOBALS ' (VARS IN GLOBALVARS OR @(GETPROP
X 'GLOBALVAR)))
```

would allow you to refer with the single word `GLOBALS` to the set of variables which are either in `GLOBALVARS` or have a `GLOBALVAR` property.

Functions for Writing Routines

The following functions are provided for users who wish to write their own routines using MasterScope's database:

(`PARSERELATION` *RELATION*) [Function]

RELATION is a relation phrase; e.g., (`PARSERELATION` ' (USE FREELY)).
`PARSERELATION` returns an internal representation for *RELATION*. For use in conjunction with `GETRELATION`.

(`GETRELATION` *ITEM RELATION INVERTED*) [Function]

RELATION is an internal representation as returned by `PARSERELATION` (if not, `GETRELATION` first performs (`PARSERELATION` *RELATION*)).

ITEM is an atom. `GETRELATION` returns the list of all atoms which have the given relation to *ITEM*.

For example,

(`GETRELATION` 'X ' (USE FREELY))

returns the list of variables that X uses freely.

If *INVERTED* is T, the inverse relation is used; e.g.

(`GETRELATION` 'X ' (USE FREELY) T)

returns the list of functions which use x freely.

If *ITEM* is NIL, `GETRELATION` returns the list of atoms which have *RELATION* with *any* other item; i.e., it answers the question WHO *RELATIONS* ANY.

Note that `GETRELATION` does not check to see if *ITEM* has been analyzed, or that other functions that have been changed have been reanalyzed.

(`TESTRELATION` *ITEM RELATION ITEM2 INVERTED*) [Function]

Is equivalent to (`MEMB` *ITEM2* (`GETRELATION` *ITEM RELATION INVERTED*)); that is, it tests if *ITEM* and *ITEM2* are related via *RELATION*.

If *ITEM2* is NIL, the call is equivalent to

(`NOT` (`NULL` (`GETRELATION` *ITEM RELATION INVERTED*)))

i.e., `TESTRELATION` tests if *ITEM* has the given *RELATION* with any other item.

(`MAPRELATION` *RELATION MAPFN*) [Function]

Calls the function *MAPFN* on every pair of items related via *RELATION*. If (`NARGS` *MAPFN*) is 1, then *MAPFN* is called on every item which has the given *RELATION* to *any* other item.

(`MSNEEDUNSAVE` *FNS MSG MARKCHANGEFLG*) [Function]

Used to mark functions which depend on a changed record declaration (or macro, etc.), and which must be LOADED or UNSAVED (see below). *FNS* is a list of

functions to be marked, and *MSG* is a string describing the records, macros, etc. on which they depend. If *MARKCHANGEFLG* is non-NIL, each function in the list is marked as needing reanalysis.

(UPDATEFN *FN* *EVENIFVALID* —) [Function]

Equivalent to the command `ANALYZE 'FN`; that is, UPDATEFN analyzes *FN* if *FN* has not been analyzed before or if it has been changed since the time it was analyzed. If *EVENIFVALID* is non-NIL, UPDATEFN reanalyzes *FN* even if MasterScope thinks it has a valid analysis in the database.

(UPDATECHANGED) [Function]

Performs (UPDATEFN *FN*) on every function which has been marked as changed.

(MSMARKCHANGED *NAME* *TYPE* *REASON*) [Function]

Mark that *NAME* has been changed and needs to be reanalyzed. See MARKASCHANGED in the *IRM*.

(DUMPDATABASE *FNLST*) [Function]

Dumps the current MasterScope database on the current output file in a LOADable form. If *FNLST* is not NIL, DUMPDATABASE only dumps the information for the list of functions in *FNLST*. The variable DATABASECOMS is initialized to

```
((E (DUMPDATABASE)))
```

Thus, you may merely perform (MAKEFILE 'DATABASE.*EXTENSION*) to save the current MasterScope database. If a MasterScope database already exists when a DATABASE file is loaded, the database on the file is merged with the one in memory.

Note: Functions whose definitions are different from their definition when the database was made must be REANALYZED if their new definitions are to be noticed.

Note: The DataBaseFns library module provides a more convenient way of saving databases along with the source files to which they correspond.

Noticing Changes that Require Recompiling

When a record declaration, iterative statement operator or macro is changed, and MasterScope has noticed a use of that declaration or macro (i.e., it is used by some function known about in the database), MasterScope alerts you about those functions which might need to be recompiled (e.g., they do not currently have *EXPR* definitions). Extra functions may be noticed.

For example, if *FOO* contains (fetch (REC *X*) --), and some declaration other than REC which contains *X* is changed, MasterScope still thinks that *FOO* needs to be loaded/unsaved. The functions which need recompiling are added to the list MSNEEDUNSAVE and a message is printed out:

The functions *FN1*, *FN2*, ... use macros which have changed.

Call UNSAVEFNS() to load and/or unsave them.

In this situation, the following function is useful:

(UNSAVEFNS —)

[Function]

Uses `LOADFNS` or `UNSAVEDEF` to make sure that all functions in the list `MSNEEDUNSAVE` have `EXPR` definitions, and then sets `MSNEEDUNSAVE` to `NIL`.

Note: If `RECOMPILEDEFAULT` (see *IRM*) is set to `CHANGES`, `UNSAVEFNS` prints out

"WARNING: you must set RECOMPILEDEFAULT to EXPRS in order to have these functions recompiled automatically."

Implementation Notes

MasterScope keeps a database of the relations noticed when functions are analyzed. The relations are intersected to form primitive relationships such that there is little or no overlap of any of the primitives.

For example, the relation `SET` is stored as the union of `SET LOCAL` and `SET FREE`. The `BIND` relation is divided into `BIND AS ARG`, `BIND AND NOT USE`, and `SET LOCAL`, `SMASH LOCAL`, etc. Splitting the relations in this manner reduces the size of the database considerably, to the point where it is reasonable to maintain a MasterScope database for a large system of functions during a normal debugging session.

Each primitive relationship is stored in a pair of hash tables, one for the forward direction and one for the reverse.

For example, there are two hash tables, `USE AS PROPERTY` and `USED AS PROPERTY`. To retrieve the information from the database, MasterScope performs unions of the hash values.

For example, to answer `FOO BINDS WHO`, MasterScope looks in all of the tables which make up the `BIND` relation. The internal representation returned by `PARSERELATION` is a list of dotted pairs of hash tables. To perform `GETRELATION` requires only mapping down that list, doing `GETHASHs` on the appropriate hash tables and `UNIONING` the result.

Hash tables are used for a variety of reasons: storage space is smaller; it is not necessary to maintain separate lists of which functions have been analyzed (a special table, `DOESN'T DO ANYTHING` is maintained for functions which neither call other functions nor bind or use any variables); and accessing is relatively fast. Within any of the tables, if the hash value is a list of one atom, then the atom itself, rather than the list, is stored as the hash value. This also reduces the size of the database significantly.

Example

Sample Session

The following illustrates some of the MasterScope facilities.

```
50_. ANALYZE FUNCTIONS ON RECORD
.....
NIL
51_. WHO CALLS RECFIELDLOOK
```

```

(RECFIELDLOOK ACCESSDEF ACCESSDEF2 EDITREC)
52_. EDIT WHERE ANY CALL RECFIELDLOOK
RECFIELDLOOK :
(RECFIELDLOOK (CDR Y) FIELD)
tty:
5*OK
ACCESSDEF :
(RECFIELDLOOK DECLST FIELD VAR1)
6*OK
(RECFIELDLOOK USERRECLST FIELD)
7*N VAR1
8*OK
ACCESSDEF2 :
(RECFIELDLOOK (RECORD.SUBDECS TRAN) FIELD)
tty:
(RECFIELDLOOK (RECORD.SUBDECS TRAN) FIELD)
9*N (CAR TAIL]
10*OK
EDITREC :
(RECFIELDLOOK USERRECLST (CAR EDITRECX))
11*OK
NIL
53_. WHO CALLS ERROR
..
(EDITREC)
54_. SHOW PATHS TO RECFIELDLOOK FROM ACCESSDEF
(inverted tree)

1. RECFIELDLOOK RECFIELDLOOK
2.                ACCESSDEF
3.                ACCESSDEF2 ACCESSDEF2
4.                ACCESSDEF
5.                RECORDCHAIN
ACCESSDEF
NIL
55_. WHO CALLS WHO IN /FNS
RECORDSTATEMENT -- /RPLNODE
RECORDECL1 --      /NCONC, /RPLACD, /RPLNODE
RECREDECLARE1 --   /PUTHASH
UNCLISPTRAN --     /PUTHASH, /RPLNODE2
RECORDWORD --      /RPLACA
RECORD1 --         /RPLACA, /SETTOPVAL
EDITREC --         /SETTOPVAL

```

Event 50 You direct that the functions on file RECORD be analyzed. The leading period and space specify that this line is a MasterScope command. MasterScope prints a greeting and prompts with `_`. Within the top-level Executive of MasterScope, you may issue MasterScope commands, programmer's assistant commands, (e.g., REDO, FIX), or run programs. You can exit from the MasterScope Executive by

typing OK. The function "." is defined as a Nlambda NoSpread function which interprets its argument as a MasterScope command, Executes the command and returns.

MasterScope prints a "." whenever it (re)analyzes a function, to let you know what it is happening. The feedback when MasterScope analyzes a function is controlled by the flag MSPRINTFLG: if MSPRINTFLG is the atom ".", MasterScope prints out a period. (If an error in the function is detected, "?" is printed instead.) If MSPRINTFLG is a number *N*, MasterScope prints the name of the function it is analyzing every *N*th function. If MSPRINTFLG is NIL, MasterScope won't print anything. Initial setting is ".".

Note that the function name is printed when MasterScope starts analyzing, and the comma is printed when it finishes.

- Event 51 You ask which functions call RECFIELDLOOK. MasterScope responds with the list.
- Statement 52 You ask to edit the expressions where the function RECFIELDLOOK is called. MasterScope calls EDITF on the functions it had analyzed that call RECFIELDLOOK, directing the editor to the appropriate expressions. You then edit some of those expressions. In this example, the teletype editor is used. If DEdit is enabled as the primary editor, it would be called to edit the appropriate functions.
- Statement 53 Next you ask which functions call ERROR. Since some of the functions in the database have been changed, MasterScope reanalyzes the changed definitions (and prints out .'s for each function it analyzes). MasterScope responds that EDITREC is the only analyzed function that calls ERROR.
- Statement 54 You ask to see a map of the ways in which RECFIELDLOOK is called from ACCESSDEF. A tree structure of the calls is displayed.
- Statement 55 You then ask to see which functions call which functions in the list /FNS. MasterScope responds with a structured printout of these relations.

SHOW PATHS

The command SHOW PATHS FROM MSPARSE prints out the structure of MasterScope's parser:

```
1.MSPARSE  MSINIT MSMARKINVALID
2.         |      MSINITH MSINITH
3.         MSINTERPRET MSRECORDFILE
4.         |      MSPRINTWORDS
5.         |      PARSECOMMAND GETNEXTWORD CHECKADV
6.         |      |      PARSERELATION {a}
7.         |      |      PARSESET {b}
8.         |      |      PARSEOPTIONS {c}
9.         |      |      MERGECONJ GETNEXTWORD {5}
10.        |      GETNEXTWORD {5}
```

```

11.      |          FIXUPTYPES SUBJTYPE
12.      |          |          OBJTYPE
13.      |          FIXUPCONJUNCTIONS MERGECONJ {9}
14.      |          MATCHSCORE
15.      MSPRINTSENTENCE
-----
overflow - a
16.PARSERELATION GETNEXTWORD {5}
17.          CHECKADV
-----
overflow - b
19.PARSESET PARSESET
20.          GETNEXTWORD {5}
21.          PARSERELATION {6}
22.          SUBPARSE GETNEXTWORD {5}
-----
overflow - c
23.PARSEOPTIONS GETNEXTWORD {5}
24.          PARSESET {19}

```

This example shows that the function MSPARSE calls MSINIT, MSINTERPRET, and MSPRINTSENTENCE. MSINTERPRET in turn calls MSRECORDFILE, MSPRINTWORDS, PARSECOMMAND, GETNEXTWORD, FIXUPTYPES, and FIXUPCONJUNCTIONS. The numbers in braces {} after a function name are backward references: they indicate that the tree for that function was expanded on a previous line. The lowercase letters in braces are forward references: they indicate that the tree for that function will be expanded below, since there is no more room on the line. The vertical bar is used to keep the output aligned.

[This page intentionally left blank]

